IT DEPARTMENT / WEB DEVELOPMENT

Laravel Authentication Lab

Class date: 19 October 2026 **Suggested time:** 120 minutes

Student name: _Ahmad Wali_ **Father name:** _Azizullah_

Your assignment: Build a small IT Student Portal. A visitor can create an account and sign in. Only signed-in users can open the dashboard. A logout button ends the session. Complete every TODO in this handout.

What you will practice

Connect Blade forms to named routes; validate registration data; store a hashed password; use Laravel session authentication; protect a page with middleware.

1. Prepare the project | 15 minutes

Use a prepared Laravel 12 project, or run the first two commands to create a separate lab project. Requirements: PHP 8.2+, Composer, MySQL, and a code editor.

TERMINAL - NEW PROJECT ONLY

```
composer create-project laravel/laravel it-auth-lab "^12.0"
cd it-auth-lab
```

Start MySQL and create an empty database named **it_auth_lab** using phpMyAdmin. Update these values in **.env**; use your own MySQL username and password if different.

.env - LOCAL LAB SETTINGS

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=it_auth_lab
DB_USERNAME=root
DB_PASSWORD=
SESSION_DRIVER=file
```

TERMINAL - INSIDE YOUR PROJECT

```
php artisan config:clear
php artisan migrate
php artisan make:controller AuthController
php artisan serve
```

Keep the server running. Use another terminal for later Artisan commands. Keep the default **User** model and **users** migration. Open **http://127.0.0.1:8000**.

TASKS 2-3 / 15 MINUTES

Connect the routes and views

2. Add the named routes

In this dedicated lab project, put the following code in **routes/web.php**. It connects the forms to the controller methods you will complete on pages 4-5.

routes/web.php

```
<?php

use App\Http\Controllers\AuthController;
use Illuminate\Support\Facades\Route;

Route::redirect('/', '/login');

Route::middleware('guest')->group(function () {
    Route::view('/register', 'auth.register')->name('register');
    Route::post('/register', [AuthController::class, 'register'])
        ->name('register.store');

    Route::view('/login', 'auth.login')->name('login');
    Route::post('/login', [AuthController::class, 'login'])
        ->name('login.store');
});

Route::middleware('auth')->group(function () {
    Route::get('/dashboard', function () {
        return view('dashboard');
    })->name('dashboard');

    Route::post('/logout', [AuthController::class, 'logout'])
        ->name('logout');
});
```

Route naming: Attach **->name('dashboard')** to the route, after the closure. The **view('dashboard')** call returns the Blade view. The **auth** middleware protects the dashboard; **guest** is for visitors who have not signed in.

3. Create these view files

Location	Purpose
resources/views/auth/register.blade.php	Registration form
resources/views/auth/login.blade.php	Login form
resources/views/dashboard.blade.php	Protected student dashboard

Create the **auth** folder manually if it does not exist. Run **php artisan route:list** and confirm the six named routes above appear.

TASK 4 / 20 MINUTES

Build the registration and login forms

4. Create the form inputs

Create complete HTML documents for both forms. Use the registration starter below inside the document body, then complete the missing fields. Give each input a visible label.

Form	Input name	HTML type / requirement
Register	name	text; keep old('name')
Register	email	email; keep old('email')
Register	password	password; at least 8 characters
Register	password_confirmation	password; must match password
Login	email and password	email and password; keep old email only

REGISTER FORM - STARTER

```
<h1>Create an account</h1>
<form method="POST" action="{{ route('register.store') }}">
    @csrf
    <label for="name">Full name</label>
    <input id="name" name="name" type="text"
           value="{{ old('name') }}" required>
    <!-- TODO: add email, password, password_confirmation -->
    <button type="submit">Register</button>
</form>
<a href="{{ route('login') }}">Already have an account?</a>
```

For the login form, use **method="POST"**, **route('login.store')**, **@csrf**, an email field, a password field, and a Login button. Add a link to **route('register')**.

Show feedback above each form

BLADE FEEDBACK - ADD TO BOTH FORM PAGES

```
@if ($errors->any())
    <ul>
        @foreach ($errors->all() as $error)
            <li>{{ $error }}</li>
        @endforeach
    </ul>
@endif
@if (session('success'))
    <p>{{ session('success') }}</p>
@endif
```

Keep passwords empty when redisplaying forms. The HTML **required** attribute helps in the browser; you must also validate inputs in the controller. Every POST form, including Logout, must contain **@csrf**.

TASK 5 / 20 MINUTES

Implement account registration

5. Complete the controller

Replace the generated controller with this starter. Fill in the TODOs. On page 5, complete the two empty methods inside the same class.

app/Http/Controllers/AuthController.php - STARTER

```
<?php
namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
class AuthController extends Controller
{
    public function register(Request $request)
    {
        $data = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|max:255|unique:users,email',
            'password' => 'required|string|min:8|confirmed',
        ]);
        // TODO: create a User and set name, email, password.
        // TODO: save the user and redirect to the login page.
    }

    public function login(Request $request)
    {
        // TODO: complete using page 5.
    }

    public function logout(Request $request)
    {
        // TODO: complete using page 5.
    }
}
```

Registration requirements

a. Create a model with **$user = new User();** and assign its **name** and **email** from **$data**.

b. Set the password using **$user->password = Hash::make($data['password']);** and call **$user->save();**

c. Return **redirect()->route('login')->with('success', 'Account created. Please log in.');**

The **confirmed** rule checks **password_confirmation**. Do not store the confirmation field. After a valid registration, inspect the users table: the stored password must be a hash, rather than the password you typed.

TASKS 6-8 / 30 MINUTES

Sign in, show the dashboard, and log out

6. Complete login(Request $request)

Use this method body. First replace the TODO with **$request->validate(...)** assigned to **$credentials**: email must be **required|email** and password must be **required|string**.

BODY OF login() - COMPLETE THE TODO

```
// TODO: validate email and password into $credentials.

$accepted = Auth::attempt($credentials);
if (! $accepted) {
    return back()->withErrors([
        'email' => 'Email or password is incorrect.',
    ])->onlyInput('email');
}

$request->session()->regenerate();
return redirect()->intended(route('dashboard'));
```

Pass the entered password to **Auth::attempt()**. Laravel checks it against the stored hash. After success, regenerate the session and redirect to the intended page, falling back to the dashboard.

7. Create the protected dashboard

Place this content inside the body of **dashboard.blade.php**. Display the actual signed-in user, so different accounts show different details.

resources/views/dashboard.blade.php - BODY

```
<h1>IT Student Portal</h1>
<p>Welcome, {{ auth()->user()->name }}.</p>
<p>Email: {{ auth()->user()->email }}</p>

<form method="POST" action="{{ route('logout') }}">
    @csrf
    <button type="submit">Logout</button>
</form>
```

8. Complete logout(Request $request)

Write these calls in order inside the method:

BODY OF logout()

```
Auth::logout();
$request->session()->invalidate();
$request->session()->regenerateToken();
return redirect()->route('login');
```

Checkpoint: Register an account, log in, confirm your name appears, and click Logout. Then type **/dashboard** into the address bar again. You should return to the login page.

TASK 9 / 20 MINUTES

Test and demonstrate your application

9. Complete this checklist

Use test accounts and record Pass or Fix for each row. Finish the core behaviour before adding visual styling. Demo the working portal to your instructor during class.

Test	Expected result	Pass / Fix
Open /dashboard while logged out	Redirect to the login page.	_____
Register with valid, new details	One new user; login page shows success.	_____
Register an email that already exists	Validation error; no additional user.	_____
Use a short password, then mismatched confirmation	Both cases fail; no account is created.	_____
Log in with correct details	Dashboard shows the current name and email.	_____
Log in with an incorrect password	Error message; dashboard remains protected.	_____
Open /login while already logged in	Redirect to the dashboard.	_____
Log out; then request /dashboard again	Redirect to login; access requires a new login.	_____
Log in as a second test account	Dashboard shows the second account.	_____

Assessment | 10 marks

Criterion	Marks
Routes and forms, including names and CSRF tokens	2
Registration validation and hashed password storage	3
Successful login and incorrect-password handling	2
Protected dashboard and working logout	2
Testing demonstration and explanation	1

Be ready to explain

1. Why do we use **Hash::make()** when saving a new password?
2. How does **auth** middleware protect the dashboard?
3. What does **@csrf** add to a form?

Quick fixes

Route [dashboard] not defined: check the route name placement on page 2 and run **php artisan route:clear**.
419 Page Expired: check @csrf, reload the form, and use the same host throughout.
Database connection error: start MySQL, check .env, and run **php artisan config:clear**.

Official Laravel 12 references

Authentication | Validation | Routing | CSRF protection